

Reproducible runs: seeds, environments, Docker

Someone else, or you next month, should be able to rerun the code and get the same number.

THE QUESTION

Why does the same code print 92% today, 85% tomorrow and an error on a friend's laptop, and what do you ship so it prints one number everywhere?

THE INTUITION

A run depends on its random draws, its package versions, the Python and system libraries under them, and its settings. Seeds fix the draws, a pinned venv the packages, a Docker image the machine, a log the rest.

KEY TAKEAWAY

Seed every generator, pin exact versions with pip freeze, copy requirements before code in the Dockerfile, and report mean $\pm$ std over seeds.

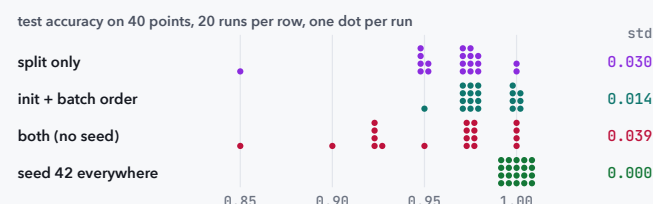
random sources $\rightarrow$ seed them $\rightarrow$ mean $\pm$ std over seeds $\rightarrow$ log the run $\rightarrow$ venv + pip freeze $\rightarrow$ Dockerfile $\rightarrow$ image

1 Five layers decide the number a run prints

Layer	fixed by	symptom when it is missing
Randomness	seeds, random_state=42	92% today, 85% tomorrow
Packages	venv + requirements.txt	ModuleNotFoundError
Machine	Docker image	OSError: a system library is missing
Memory	a tracker, meta.json	which of 50 runs was the good one?
Code	git (see the git sheet)	which version made this number?

For a small project, seeds, a venv and a tracker are usually enough. Add Docker when the code must run on another operating system or a server.

2 The split, the init and the data order each move the score



```
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42)
model = RandomForestClassifier(
    n_estimators=100, random_state=42)
kf = KFold(n_splits=5, shuffle=True,
           random_state=42)
```

Dropout is a fourth source. In a small numpy network on 200 rows, the split alone spreads accuracy from 0.85 to 1.00. Without random_state, KFold(shuffle=True) reshuffles on every call; the lab's five CV means ran from 0.954 to 0.965.

3 An unseeded call draws from NumPy's global generator

draws from NumPy's global generator, in call order	accuracy
seed(42) X = rand(200, 5) split init batch order	0.925
seed(42) X = rand(200, 5) +1 rand() split init batch order	0.950
random_state=42 in each call: the extra draw changes nothing	1.000

row 1 repeats on every run; row 2 moves the split, init and batches

With random_state=None, scikit-learn draws from the generator that np.random.seed sets, so a seed at the top already fixes the run. The score then depends on call order: one extra draw before the split changes every later draw. Pass random_state so each call has its own stream.

4 One set_seed call covers the global generators

	random.randint	np.random.randint	
seed 42, run 1	81 14 3 94 35	51 92 14 71 60	
seed 42, run 2	81 14 3 94 35	51 92 14 71 60	same
seed 7	41 19 50 83 6	47 68 25 67 83	differs

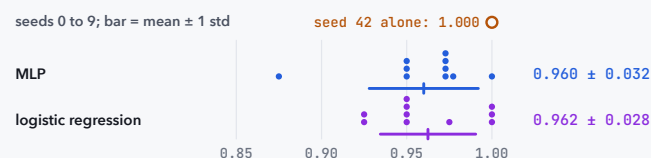
seed 42 twice: identical draws; seed 7: another repeatable stream

```
def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
```

```
set_seed(42) # once, at the top
cudnn = torch.backends.cudnn
cudnn.deterministic = True # slower
cudnn.benchmark = False # no autotuner
```

Each library keeps its own generator. The cuDNN flags make GPU convolutions repeatable at some cost in speed. Any number works; 42 is tradition.

5 A seed makes a run repeatable, not representative



Seed 42 gives this network 1.000; seeds 0 to 9 give 0.960 ± 0.032 . The two models average 0.960 and 0.962, yet each wins on 3 of the 10 seeds. BLAS libraries, package versions and GPU kernels still shift results across machines, so report mean $\pm$ std over seeds, never the best seed.

6 Store what you ran next to what you got

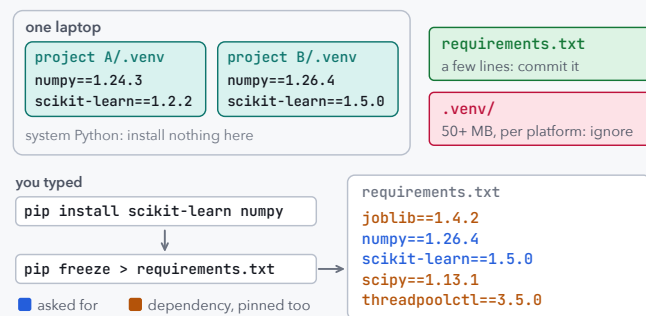
commit	3f9c2e1 (illustrative id)
environment	requirements.txt · image train-repro
python	3.10 (FROM python:3.10-slim)
data	200 rows, 5 features, 40 test points
config	MLP 5-8-1, 15 epochs, lr 0.5, batch 16
seeds	0 to 9, split and model
result	accuracy 0.960 ± 0.032 over 10 seeds

The lab's capstone writes accuracy, seed, scikit-learn version and timestamp to outputs/meta.json beside the model. A tracker (trackio.init, log, finish; see the tuning and tracking sheet) keeps config and metrics per run.

Pin the packages, then ship the machine

pip freeze fixes the Python packages. A Docker image also fixes Python itself, the OS and the system libraries.

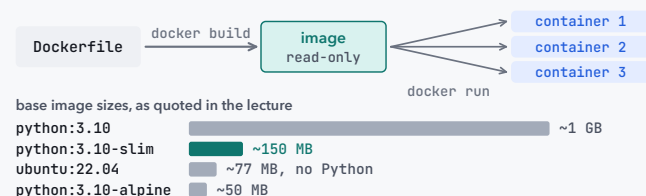
7 A venv per project, pinned with pip freeze



```
python -m venv .venv          # create
source .venv/bin/activate     # enter
# Windows: .venv\Scripts\activate
pip install scikit-learn numpy
pip freeze > requirements.txt
deactivate                    # leave
# elsewhere, in a fresh venv:
pip install -r requirements.txt
```

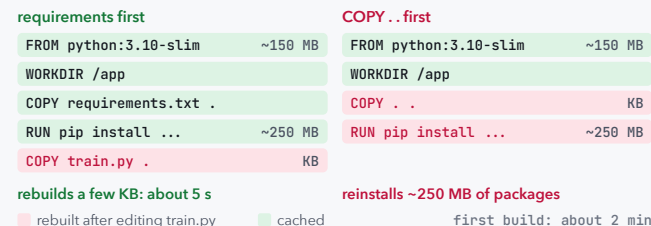
A venv reuses the Python it was made with and leaves the OS and system libraries alone. pip freeze pins every installed package, even ones you never named, so it acts as a lock file; a file with only scikit-learn==1.7.2 leaves numpy and scipy free. Conda's environment.yml also pins Python.

8 An image is built once; each container is one run of it



One image gives many disposable containers, as a class gives objects. Containers share the host's kernel, so they weigh MBs and start in seconds; a VM boots a whole guest OS. On a Mac or Windows, Docker Desktop runs one Linux VM underneath, hence "Linux" in the demo. -alpine can break packages that need glibc.

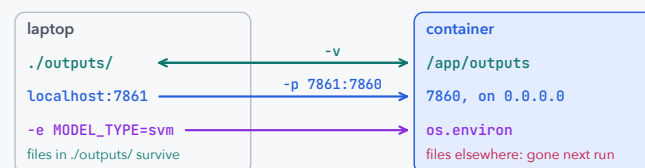
9 Copy requirements before code so edits skip pip install



```
FROM python:3.10-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir \
    -r requirements.txt
COPY train.py .
CMD ["python", "train.py"]
```

A changed layer rebuilds itself and every layer after it. Keep comments on their own lines (after an instruction, # is read as an argument), and list .venv/ and .git/ in .dockerignore.

10 Containers forget: mount a volume, map a port, pass settings



```
docker build -t train-save .
docker run \
    -v $(pwd)/outputs:/app/outputs train-save
docker run -p 7861:7860 spam-app
docker run -e MODEL_TYPE=svm env-demo
docker run -it train-save bash # a shell
```

Each docker run starts a fresh container. Files written inside stay in that stopped container until docker rm, out of the next run's sight; -v writes them to the laptop. A web app must listen on 0.0.0.0 (ENV GRADIO_SERVER_NAME="0.0.0.0"). Read -e settings with os.environ.get("MODEL_TYPE", "rf").

11 Pause and think

No module named 'pandas'? The fresh venv skipped pip install -r requirements.txt.

A 3 GB image? Use FROM python:3.10-slim; docker system prune clears old images.

0.847 here, 0.831 there, same venv? No seed in the split or the model.

port is already allocated? Use another laptop port: -p 7862:7860.

libgomp.so.1: cannot open shared object file? A missing system library; an image can ship it.

Container stopped. Weights? Inside it until docker rm; docker cp gets them. Next time, -v.

seed every generator · mean ± std over seeds · log the run · pip freeze · requirements before code · -v for outputs

Source: lectures "Reproducibility in practice" and "Reproducibility and environments", the reproducibility lab, the Docker demos and the reproducibility follow-along, nipunbatra.github.io/stt-ai-teaching. Accuracies are numpy computations with fixed seeds on the follow-along's toy data (200 rows, 40 test points); a 5-8-1 network and a logistic regression stand in for the scikit-learn models. Image sizes, build times, pip freeze lines and the lab's CV range are quoted; the 250 MB package layer is the follow-along's 400 MB image minus the 150 MB base (all approximate); the commit id is illustrative.